Practice · Whole programs 5

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the marks program does

Read the whole program before you answer.

```
#include <stdio.h>

#define N 6

double average(int a[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += a[i];
    }
    return (double) sum / n;
}

int index_of_max(int a[], int n) {
    int best = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] > a[best]) {
            best = i;
        }
    }
    return best;
}

int count_above(int a[], int n, double limit) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] > limit) {
            count++;
        }
    }
    return count;
}

int main(void) {
    int marks[N] = {72, 95, 61, 88, 79, 54};
    double avg = average(marks, N);
    int top = index_of_max(marks, N);
    printf("average: %.1f\n", avg);
    printf("best: student %d with %d\n", top + 1, marks[top]);
    printf("above average: %d\n", count_above(marks, N, avg));
    return 0;
}
```

What does the whole program do?

- It sorts six marks and prints them.
- **Right answer:** For six marks, it prints the average, who has the best mark, and how many marks are above the average.
- It reads six marks from the keyboard and prints the largest.
- It prints every mark that is above 60.

Feedback: `main` reads like a list of steps, and each step is a function that gets the array and its size: `average`, `index_of_max`, `count_above`. Class 10, slides 7 to 11 and 66 to 67.

(mc) L1 · Why top + 1

Same program as the question before. `index_of_max` returns 1 for these marks. Why does `main` print `top + 1`?

- **Right answer:** Because indexes start at 0, but people count students from 1.
- Because `index_of_max` returns one too few by mistake.
- Because `marks` has `N + 1` elements.
- Because `%d` prints one less than the value.

Feedback: 95 is at index 1, the second element. People say "student 2", so the program adds 1 for the reader. The index itself is right. Class 9 (indexes from 0); class 10, slides 41 and 42.

(mc) L2 · The marks program prints

Same program as before. What are the three lines it prints?

- `average: 74.8`, `best: student 1 with 95`, `above average: 3`
- **Right answer:** `average: 74.8`, `best: student 2 with 95`, `above average: 3`
- `average: 74.0`, `best: student 2 with 95`, `above average: 2`
- `average: 74.8`, `best: student 2 with 95`, `above average: 6`

Feedback: The sum is 449, and 449 / 6 is 74.83, printed as `74.8` with `%.1f`. The best mark, 95, is at index 1: student 2. Three marks are above 74.8: 95, 88 and 79. Class 10, slides 10, 11, 41 and 66 to 67.

(mc) L2 · Find the bug: average 74.0

Someone changes one line of `average` and now the program prints `average: 74.0`. Which line did they write?

- `int sum = 1;`
- `for (int i = 1; i < n; i++)`
- **Right answer:** `return sum / n;`
- `return (double) (sum / n) + 0.5;`

Feedback: `sum` and `n` are both `int`, so `sum / n` is whole-number division: 449 / 6 is 74. Turning that into a `double` gives `74.0`. The cast must come first, `(double) sum / n`, so the division is done with decimals. Class 10, slide 11.

(fib) L3 · Complete average: keep the decimals

Same program as before. Complete the `return` of `average` so the division keeps its decimals. Type only what goes before `sum`, with its brackets.

```
return [(double)|( double )] sum / n;
```

Feedback: The cast `(double)` turns `sum` into a `double` before the division, so 449 / 6 gives 74.83, not 74. Class 10, slide 11; class 3 for casts.

(mc) L1 · What the sort program does

Read the whole program before you answer.

```
#include <stdio.h>

void bubble_sort(int a[], int n) {
    for (int pass = 1; pass < n; pass++) {
        for (int j = 0; j < n - pass; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        }
    }
    return -1;
}

int main(void) {
    int a[5] = {40, 10, 50, 30, 20};
    bubble_sort(a, 5);
    for (int i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("30 is at %d\n", find(a, 5, 30));
    printf("35 is at %d\n", find(a, 5, 35));
    return 0;
}
```

`bubble_sort` returns nothing. How does `main` get the sorted numbers?

- It cannot: `main`'s array stays `40 10 50 30 20`.
- **Right answer:** An array is not copied: `bubble_sort` sorts `main`'s own array.
- `bubble_sort` prints them itself.
- The last line of `bubble_sort` is a hidden `return a;`.

Feedback: When you send an array to a function, the function works on the caller's own elements. So after the call, `main`'s `a` is sorted. An `int` parameter would be a copy (class 8); an array is not. Class 10, slides 12 to 15 and 61 to 62.

(mc) L2 · The sort program prints

Same program as before. What are the three lines it prints?

- `40 10 50 30 20`, `30 is at 3`, `35 is at -1`
- `10 20 30 40 50`, `30 is at 3`, `35 is at 0`
- **Right answer:** `10 20 30 40 50`, `30 is at 2`, `35 is at -1`
- `10 20 30 40 50`, `30 is at 30`, `35 is at -1`

Feedback: After the sort the array is `10 20 30 40 50`, so 30 is at index 2. `find` returns the index, not the value. 35 is not there: the loop looks at all five, then `return -1;` after the loop. Class 10, slides 24 and 56 to 58.

(fib) L3 · Complete find: not found

Same program as before. Type the number `find` returns when the target is not in the array.

```
    return [-1];
}
```

Feedback: -1 is never a real index, because indexes start at 0, so it is a safe signal for "not found". It goes **after** the loop, when every element has been checked. Class 10, slides 21 to 24.

(fib) L3 · Complete bubble_sort: the passes

Same program as before. An array of `n` numbers needs `n - 1` passes. Type what goes after `pass <` so that `pass` takes the values 1 to `n - 1`.

```
for (int pass = 1; pass < [n]; pass++) {
```

Feedback: `pass` starts at 1 and the loop runs while `pass < n`, so for 5 numbers it makes passes 1, 2, 3 and 4: four passes. After them every number is in place. Class 10, slides 53 to 57.

(mc) L1 · What the word program does

Read the whole program before you answer.

```
#include <stdio.h>
#include <string.h>

int count_char(char s[], char c) {
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) {
            count++;
        }
    }
    return count;
}

void reverse(char s[]) {
    int i = 0;
    int j = strlen(s) - 1;
    while (i < j) {
        char temp = s[i];
```

```
        s[i] = s[j];
        s[j] = temp;
        i++;
        j--;
    }
}

int main(void) {
    char word[20] = "banana";
    char back[20];
    strcpy(back, word);
    reverse(back);
    printf("%s backwards is %s\n", word, back);
    printf("letters: %d, a: %d\n", (int) strlen(word), count_char(word, 'a'));
    if (strcmp(word, back) == 0) {
        printf("a palindrome\n");
    } else {
        printf("not a palindrome\n");
    }
    return 0;
}
```

Why does `main` copy `word` into `back` before it calls `reverse`?

- Because `reverse` cannot work on an array called `word`.
- **Right answer:** Because `reverse` changes the array it gets, and `main` still needs the original word.
- Because `strcmp` only works on copies.
- It is not needed: `reverse` works on a copy anyway.

Feedback: An array is not copied when it goes to a function, so `reverse(word)` would turn `word` itself around. Reversing a copy keeps both: `word` forwards and `back` backwards, ready to compare. Class 12, slides 14 and 32 to 34.

(mc) L2 · The word program prints

Same program as before. What does it print?

- `banana backwards is banana`, `letters: 6, a: 3`, `a palindrome`
- `banana backwards is ananab`, `letters: 7, a: 3`, `not a palindrome`
- `banana backwards is ananab`, `letters: 6, a: 2`, `not a palindrome`
- **Right answer:** `banana backwards is ananab`, `letters: 6, a: 3`, `not a palindrome`

Feedback: Backwards, `banana` is `ananab`. It has 6 letters (`strlen` does not count `'\0'`) and three as. The two strings differ in the first letter, so `strcmp` is not 0. Class 12, slides 9, 28 and 32 to 34.

(mc) L2 · Find the bug: level is not a palindrome

Someone changes the `if` to `if (word == back)` and the word to `"level"`. The program now prints `not a palindrome`. Why?

- `reverse` does not work on words with an odd number of letters.
- `"level"` backwards is `"levle"`.
- **Right answer:** `==` asks whether `word` and `back` are the same array, not whether they have the same letters.

- `strcpy` did not copy the word.

Feedback: `word` and `back` are two different arrays, so `word == back` is always false, whatever letters they hold. gcc warns: `comparison between two arrays`. Comparing the letters needs `strcmp(word, back) == 0`, and then `level` is a palindrome. Class 12, slides 23 to 28.

(fib) L3 · Complete the call: count the a's

Same program as before. Complete the call so that it counts the letter a in `word`. Type only the second argument.

```
count_char(word, ['a'])
```

Feedback: One character goes in single quotes: `'a'`. Double quotes, `"a"`, would make a string, which is an array, not a `char`. Class 11, slides 22 to 24.

(mc) L1 · What the adding program does

Read the whole program before you answer.

```
#include <stdio.h>
#include <string.h>

int to_int(char s[]) {
    int n = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        n = n * 10 + (s[i] - '0');
    }
    return n;
}

int main(void) {
    char a[10] = "125";
    char b[10] = "75";
    int sum = to_int(a) + to_int(b);
    char text[20];
    sprintf(text, "%d", sum);
    printf("%s + %s = %s\n", a, b, text);
    printf("the answer has %d digits\n", (int) strlen(text));
    char joined[20];
    strcpy(joined, a);
    strcat(joined, b);
    printf("joined: %s\n", joined);
    return 0;
}
```

What does `to_int` do?

- It counts the characters of a string.
- It turns a number into a string.
- **Right answer:** It turns a string of digits, like `"125"`, into the number 125.
- It adds up the digits: `"125"` gives 8.

Feedback: Each round multiplies what it has by 10 and adds the next digit: 1, 12, 125. It does by hand what `atoi` does. Class 12, slides 39 to 43.

(mc) L2 · The adding program prints

Same program as before. What are the three lines it prints?

- **Right answer:** `125 + 75 = 200`, `the answer has 3 digits`, `joined: 12575`
- `125 + 75 = 12575`, `the answer has 5 digits`, `joined: 200`
- `125 + 75 = 200`, `the answer has 4 digits`, `joined: 12575`
- `125 + 75 = 200`, `the answer has 3 digits`, `joined: 200`

Feedback: The numbers add: 125 + 75 = 200. `sprintf` turns 200 into the text `"200"`, 3 characters. `strcat` joins **text**, so the strings `"125"` and `"75"` become `"12575"`. Adding needs numbers; joining needs strings. Class 12, slides 16, 42 and 47.

(mc) L2 · Room for joined

Same program as before. Which size of `joined` is the smallest that still works?

- `char joined[5];`
- `char joined[3];`
- Any size works, because `strcat` makes the array bigger.
- **Right answer:** `char joined[6];`

Feedback: `"12575"` is 5 characters, plus one box for `'\0'`: 6. C never makes an array bigger, and `strcat` does not check: a smaller array is written past its end. Class 12, slides 18 to 21.

(fib) L3 · Complete the digits line

Same program as before. Complete the call that counts the characters of `text`. Type only the function name.

```
printf("the answer has %d digits\n", (int) [strlen](text));
```

Feedback: `strlen` counts the characters before `'\0'`. `"200"` has 3. Class 12, slide 9.

(fib) L3 · Complete the conversion back to text

Same program as before. Complete the call that writes the number `sum` into the array `text`. Type only the function name.

```
[sprintf](text, "%d", sum);
```

Feedback: `sprintf` works like `printf`, but it writes into the array given first. `snprintf(text, sizeof(text), "%d", sum)` is the safe version. Class 12, slide 47.